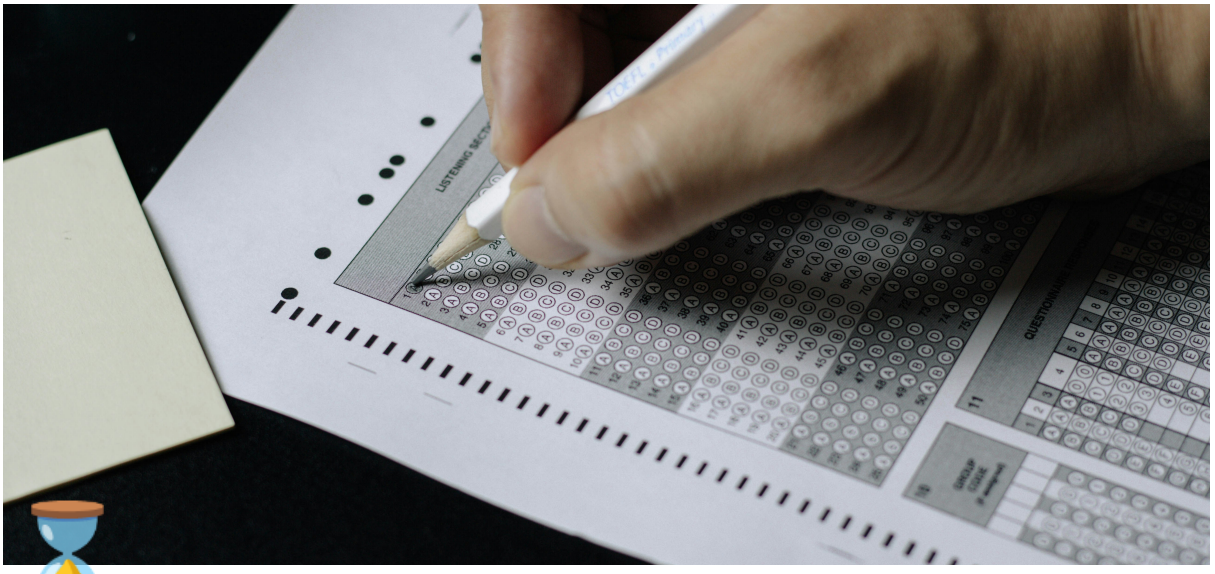


⏳

Examen 2025 repetido

Ejercicio 1

Se tienen los procesos P1,P2 y P3 con sus instrucciones originales. Se desea modificar el comportamiento de forma que:

- P2 solo pueda ejecutar ‘e’ después de que P1 haya ejecutado ‘b’ al menos una vez, y P3 haya ejecutado ‘h’ exactamente una vez.
- P2 no debe bloquearse indefinidamente si P3 ejecuta ‘h’ antes que P1
- Solo pueden usarse semáforos POSIX no nombrados

```
Proceso P1;
begin
    while true do
    begin
        a;
        b;
        c;
    end;
end;

Proceso P2;
begin
```

```
    while true do
    begin
        d;
        e;
        f;
    end;
end;

Proceso P3;
begin
    while true do
    begin
        g;
        h;
        i;
    end;
end;
```

a) Diseñar la solución sin interbloqueo

```
Semaphore b, h;
sem_init(b,0);
sem_init(h,1);

Proceso P1;
begin
    while true do
    begin
        a;
        b;
        sem_signal(b);
        c;
    end;
end;

Proceso P2;
begin
    while true do
```

```
    begin
        d;
        sem_wait(b);
        e;
        sem_signal(h);
        f;
    end;
end;

Proceso P3;
begin
    while true do
    begin
        g;
        sem_wait(h);
        h;
        i;
    end;
end;
```

b) Demostrar que no hay inanición para P2 aunque P1 sea más lento

Es imposible que en el programa anterior se produzca inanición para P2 aunque P1 sea más lento. Se ejecute cuando se ejecute P1, en el momento en el que P2 llegue al wait que hay antes de e, se quedará esperando a que P1 realice el signal trás ejecutar b, de esta forma, no importa cuanto tarde P1, mientras que termine ejecutando b.

Ejercicio 2

Considérese el siguiente programa concurrente, donde cada instrucción entre < y > se ejecuta de forma atómica

```
<x==0 ∧ y==0 ∧ n>0>

cobegin
    <x=x+1> ||
    while (y<n) do
```

```
    begin
        <y=y+1>;
    end
coend
```

Se pide:

a) Determinar si el triple de Hoare “<x==0 ⋀ y==0 ⋀ n>0>P{x==1 ⋀ y==n>” concurrente es demostrable usando la Lógica de Programas (incluyendo las reglas del while y la regla de composición concurrente).

Primer triple:

```
<x=0> x=x+1 <x=1>
```

Segundo triple

```
<y=0 ⋀ n>0> while(y<n) <y=n>
```

Como en ambos triples, las precondiciones y postcondiciones no interfieren, ya que cada variable se modifica en un único triple, aplicando las reglas de la concurrencia podemos afirmar que el triple

```
<x==0 ⋀ y==0 ⋀ n>0>P{x==1 ⋀ y==n>
```

Es demostrable mediante lógica de programas

b) En caso afirmativo, construir una demostración completa, indicando:

- El invariante del bucle while
- Las triples atómicas utilizadas
- Las triples resultantes de combinar cada proceso mediante: la regla de composición secuencial, la regla de composición iterativa (para el bucle), y la regla de composición concurrente.

Invariante del while: $0 \leq y \leq n$

Triples atómicas:

```
<x=0> x=x+1 <x=1>
<0<=y<=n ∧ y=0 ∧ n>0> y=y+1 <0<=y<=n>
```

Triples de procesos

```
<x=0> x=x+1 <x=1>
<y=0 ∧ n>0> while (y<n) do ⟨y:=y+1⟩ <y=n>
```

Triple global:

```
<x=0 ∧ y=0 ∧ n>0> cobegin x=x+1 || while (y<n) do ⟨y:=y+1⟩
coend <x=1 ∧ y=n>
```

c) Discutir si existe interferencia entre los dos procesos. En particular, justificar si el proceso del bucle puede invalidar las aserciones sobre x (o viceversa) según las reglas de interferencia de Hoare para concurrencia.

Dado que todas las aserciones sobre P1 no mencionan y, y todas las aserciones sobre el bucle no mencionan x, podemos afirmar que no existe interferencia entre los dos procesos.

Ejercicio 3

Se diseña una solución de exclusión mutua en memoria compartida donde pueden entrar como máxima n procesos simultáneamente en sección crítica, con n≥2.

En esta versión:

- Se debe garantizar una cola FIFO para procesos que esperan cuando el aforo está completo
- El cerrojo permite hasta n procesos dentro, pero la cola usa un único contador protegido
- Al salir un proceso, solo se desbloquea a un proceso de la cola, no a todos.

Tareas:

a) Dar pseudocódigo de entrada y salida

Problema 4

Se desea un monitor CentroRecursos para gestionar acceso de procesos de tipo 1 y tipo 2 con estas reglas:

1. Solo procesos del mismo tipo pueden estar dentro simultáneamente
2. Nunca pueden coincidir procesos de tipos distintos
3. Si hay procesos esperando de ambos tipo, tiene prioridad el tipo que haya acumulado más esperas totales
4. Al terminar, un proceso despierta a uno del tipo con mayor prioridad acumulada.

El monitor exporta: entrar(tipo), salir(tipo)

a) Diseñar el monitor con variables internas, colas de condición y contadores de espera.

```
monitor Centro{
    private:
        int max_procesos;

        int dentro;
        int tipodentro;
        condition colas[2];

    public:
        Centro(){
            max_procesos=3;

            dentro=0;
            tipodentro=-1;
        }

        void entrar(tipo){
            if ((tipodentro!=tipo && tipodentro!=-1) || den
tro==max_procesos){
                colas[tipo].wait();
            }
```

```
            tipodentro=tipo;
            dentro++;
        }

        void salir(tipo){
            dentro--;
            int otrotipo=1-tipo;

            if (colas[tipo].queue()>colas[otrotipo].queue
()){
                if (dentro<max_procesos){
                    colas[tipo].signal();
                }
            } else if (colas[tipo].queue()<colas[otrotipo].
queue()){
                if (dentro==0 || (dentro<max_procesos && ti
podentro==otrotipo)){
                    colas[otrotipo].signal();
                }
            } else {
                tipodentro=-1;
            }
        }
}
```

b) Mostrar como se aplica la prioridad dinámica

En este ejercicio, la prioridad dinámica la aplican los procesos al salir, de forma que antes de despertar al siguiente proceso, compueban en que cola hay más procesos esperando, y despiertan al siguiente proceso de la cola que más procesos esperando tenga (siempre que sea posible, sin que coincidan de dos tipos diferentes, y sin llegar a alcanzar el máximo de procesos posibles).

c) Justificar exclusión mutua, ausencia de interbloqueo y no inanición.

Exclusión mutua: La propia construcción de los monitores garantiza que solo un proceso pueda ejecutarse a la vez.

Ausencia de interbloqueo: ningún proceso mantiene recursos mientras espera, y antes de salir, todo proceso despierta al siguiente proceso a entrar

No inanición: Realmente no puede garantizarse no inanición debido a las restricciones del problema, de forma que si llegase 1 proceso de tipo 0, se ejecutase, luego llegase uno de tipo 1, esperase, y luego infinitos procesos de tipo 0, siempre va a haber esperando más procesos de tipo 0 que de tipo 1, por lo que el de tipo 1 sufriría inanición.